

INFORME DE ARQUITECTURA: PIPELINE DE INGESTA Y TRANSFORMACIÓN DE TRANSACCIONES

Proyecto Integrador: Pipeline de Datos Transaccionales (Airflow & PostgreSQL)

Integrantes: Jorge Andres Sanchez Campo

Repositorio GitHub: <https://github.com/jascampo15/proyectoairflow>

1. Caso de Negocio Elegido

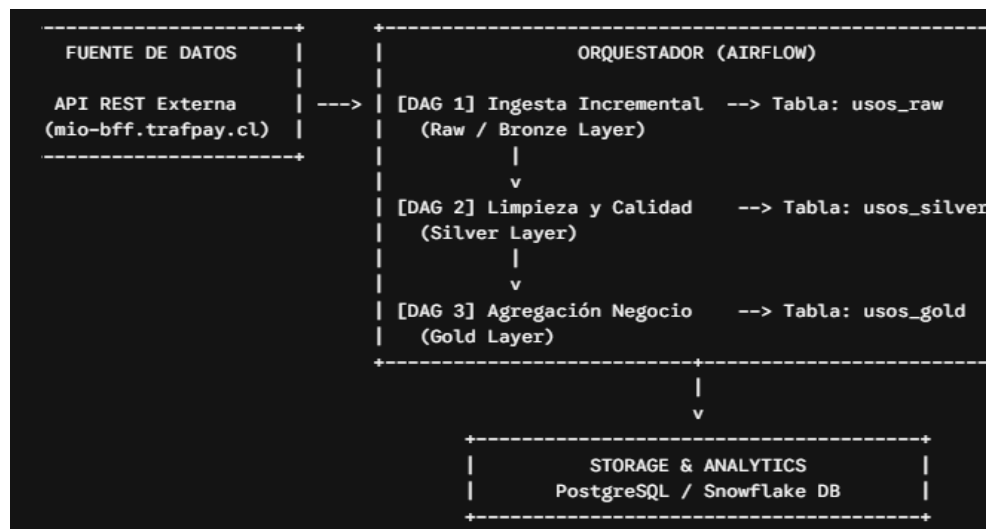
Problemática

El sistema de transporte público procesa diariamente miles de transacciones de uso de tarjetas a través de una API REST externa. Los datos crudos llegan desestructurados, inconsistencias en formatos de moneda, fechas y sin agregaciones estratégicas para la toma de decisiones.

Objetivo General

Diseñar e implementar un pipeline de datos automatizado, orquestado y escalable que ingeste datos incrementales desde la API de transacciones, los limpie y transforme en PostgreSQL aplicando arquitectura por capas (**Medallion Architecture: Raw, Silver, Gold**), garantizando seguridad, calidad de datos y un flujo de **CI/CD** continuo.

2. Diagrama de Arquitectura del Proyecto



Cuadro Resumen de Capas de Datos

Capa	Tabla	Descripción / Módulo de Transformación
Raw (Bronze)	usos_raw	Ingesta directa de payloads JSON desde API REST con deduplicación por id_txn (ON CONFLICT DO NOTHING).
Silver	usos_silver	Normalización de datos: formateo de fechas (lcl_date_time), limpieza de montos (amount), asignación de llaves operativas.
Gold	usos_gold	Agregación periódica para BI: montos totales, volumen de transacciones y promedios por fecha y línea de servicio.

3. Decisiones de Diseño e Ingeniería

A. Selección de Fuentes de Datos y Modelo de Ingesta

- **Ingesta Incremental:** Se utiliza una consulta basada en deltas tomando el MAX(id_txn) registrado en la base de datos para consumir únicamente nuevos registros. Esto optimiza el ancho de banda y evita llamados redundantes a la API.
- **Seguridad de Endpoint:** Para evitar exponer la URL del endpoint privado en repositorios públicos de GitHub, se extrae dinámicamente usando Variable.get("API_USOS_BASE_URL") configurada en la interfaz de Airflow.

B. Pruebas Automatizadas y Gobierno de Datos

- **Tests Unitarios en CI (pytest):** Priorización de pruebas sobre lógica de transformación (limpieza de montos y parseo de fechas) y carga de DAGs (DagBag import tests) para asegurar que ningún error de sintaxis bloquee la ejecución del entorno.
- **Integración Continua con GitHub Actions:** Pipeline de CI en .github/workflows/ci.yml configurado con PYTHONPATH=. para validar automáticamente el código en cada push o Pull Request previo a la mezcla con la rama main.

C. Orquestación y Modularidad

- Se diseñó un flujo desacoplado mediante TriggerDagRunOperator, asegurando que la capa **Silver** se ejecute solo tras culminar con éxito la ingesta **Raw**, previniendo carreras de datos o cargas incompletas.